

Atividade de Fixação em Sala

Autenticação com JWT no Django REST Framework — Cenários C e D

Sistema de Gerenciamento de Corridas — RunTrack

Prof. Plácido A. Souza Neto | TADS 4º Semestre

1. Contexto da Aplicação

O RunTrack é uma plataforma de gerenciamento de corridas de rua e trilha. Atletas se inscrevem em eventos, acompanham seus treinos e visualizam seus tempos. Treinadores montam planos de treino e monitoram a evolução dos atletas que orientam. Organizadores criam e administram os eventos (corridas).

A plataforma possui três perfis de usuário com responsabilidades bem distintas:

Perfil	Tipo no sistema	Permissões principais
Atleta	ATLETA	Ver eventos, se inscrever, registrar treinos próprios, ver plano de treino recebido.
Treinador	TREINADOR	Criar planos de treino, consultar atletas orientados, ver inscrições dos atletas.
Organizador	ORGANIZADOR	Criar e editar eventos (corridas), ver lista de inscritos, publicar resultados.

A atividade está dividida em duas partes. Em ambas você usará JWT (SimpleJWT) como mecanismo de autenticação, variando apenas a modelagem do perfil:

Parte	Cenário	Modelagem do perfil
Parte 1	C — Perfil por Choice + JWT	Um único modelo Usuario com campo tipo (ATLETA / TREINADOR / ORGANIZADOR).
Parte 2	D — Perfis OneToOne + JWT	Cada perfil vira uma classe própria (Atleta, Treinador, Organizador) ligada ao usuário por OneToOne.

2. Parte 1 — Cenário C: Perfil por Choice com JWT

2.1 Modelo de usuário

Crie o modelo `Usuario` herdando de `AbstractUser`. O papel do usuário na plataforma é definido pelo campo `tipo`:

backend/models.py

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class Usuario(AbstractUser):
    class Tipo(models.TextChoices):
        ATLETA = 'ATLETA', 'Atleta'
        TREINADOR = 'TREINADOR', 'Treinador'
        ORGANIZADOR = 'ORGANIZADOR', 'Organizador'

    tipo = models.CharField(max_length=12, choices=Tipo.choices,
                           default=Tipo.ATLETA)
    data_nasc = models.DateField(null=True, blank=True)
    cidade = models.CharField(max_length=80, blank=True)

    def is_atleta(self): return self.tipo == self.Tipo.ATLETA
    def is_treinador(self): return self.tipo == self.Tipo.TREINADOR
    def is_organizador(self): return self.tipo == self.Tipo.ORGANIZADOR

    def __str__(self):
        return f'{self.username} ({self.get_tipo_display()})'
```

2.2 Configuração do settings.py

Adicione as dependências e configure o JWT:

runtrack/settings.py

```
from datetime import timedelta

INSTALLED_APPS = [
    ... # apps padrão do Django
    'rest_framework',
    'rest_framework_simplejwt',
    'backend',
]

AUTH_USER_MODEL = 'backend.Usuario'

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
```

```

        'rest_framework.permissions.IsAuthenticated',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
}

```

2.3 Serializers

Crie dois serializers: um para cadastro (UsuarioSerializer) e outro customizando o login para incluir o tipo no token:

 **backend/serializers.py**

```

from rest_framework import serializers
from rest_framework_simplejwt.serializers import
TokenObtainPairSerializer
from .models import Usuario

class UsuarioSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)

    class Meta:
        model = Usuario
        fields =
['id', 'username', 'email', 'password', 'tipo', 'cidade', 'data_nasc']

    def create(self, validated_data):
        senha = validated_data.pop('password')
        usuario = Usuario(**validated_data)
        usuario.set_password(senha)
        usuario.save()
        return usuario

class LoginSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['tipo'] = user.tipo # claim extra no payload JWT
        token['username'] = user.username
        return token

    def validate(self, attrs):
        data = super().validate(attrs)
        data['tipo'] = self.user.tipo # campo extra na resposta
        return data

```

2.4 Permissões customizadas

Cada perfil terá uma classe de permissão própria, permitindo proteger endpoints de forma granular:

backend/permissions.py

```
from rest_framework.permissions import BasePermission

class IsAtleta(BasePermission):
    message = 'Apenas atletas podem realizar esta ação.'
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_atleta())

class IsTreinador(BasePermission):
    message = 'Apenas treinadores podem realizar esta ação.'
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_treinador())

class IsOrganizador(BasePermission):
    message = 'Apenas organizadores podem realizar esta ação.'
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_organizador())
```

2.5 Views

Implemente o signup (emite JWT) e três endpoints protegidos: um por perfil.

2.7 Migrações e servidor

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

2.8 Roteiro de testes — arquivo api.http

Execute as requisições abaixo em ordem. Observe os códigos HTTP e as mensagens retornadas:

api.http — Cenário C (RunTrack)

```
### 1) Cadastrar um atleta
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "pedro_corredor",
  "email": "pedro@runtrack.com",
  "password": "Trilha@123",
  "tipo": "ATLETA",
  "cidade": "Natal"
}
```

```
### 2) Cadastrar um treinador
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "coach_ana",
  "email": "ana@runtrack.com",
  "password": "Coach@456",
  "tipo": "TREINADOR",
  "cidade": "Mossoró"
}

### 3) Cadastrar um organizador
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "org_circuito",
  "email": "circuito@runtrack.com",
  "password": "Org@789",
  "tipo": "ORGANIZADOR"
}

### 4) Login do atleta (anote o access token!)
POST http://localhost:8000/api/login/
Content-Type: application/json

{ "username": "pedro_corredor", "password": "Trilha@123" }

### 5) Atleta acessa o perfil (Bearer, não Token!)
GET http://localhost:8000/api/perfil/
Authorization: Bearer <access_token_do_atleta>

### 6) Atleta registra treino (deve funcionar: 200)
POST http://localhost:8000/api/treino/
Authorization: Bearer <access_token_do_atleta>
Content-Type: application/json

{ "distancia_km": 10, "tempo_min": 58 }

### 7) Atleta tenta criar evento (deve falhar: 403)
POST http://localhost:8000/api/evento/
Authorization: Bearer <access_token_do_atleta>
Content-Type: application/json

{ "nome": "Corrida do Atleta" }

### 8) Login do organizador e criação de evento (deve funcionar: 200)
# [faça o login do org_circuito e cole o token abaixo]
POST http://localhost:8000/api/evento/
Authorization: Bearer <access_token_do_organizador>
Content-Type: application/json
```

```
{ "nome": "Circuito das Dunas 2025" }

### 9) Renovar o access token expirado
POST http://localhost:8000/api/refresh/
Content-Type: application/json

{ "refresh": "<refresh_token_do_atleta>" }
```

3. Parte 2 — Cenário D: Perfis como classes separadas com JWT

Cada tipo de usuário do RunTrack tem dados muito diferentes entre si. Nesta parte você separará essas informações em três classes de perfil próprias, ligadas ao Usuário por relacionamento OneToOne. A autenticação JWT permanece idêntica à Parte 1.

Perfil	Campos exclusivos	Exemplo de uso
Atleta	cpf, categoria (5K/10K/21K), melhor_tempo	Ver sua categoria inscrita no evento
Treinador	cref, especialidade, qtd_atletas_max	Exibir credencial na ficha do treinador
Organizador	cnpj, nome_organizacao, uf	Emitir nota fiscal do evento

3.1 Modelos de perfil

```
📄 backend/models.py — Cenário D (adicione abaixo de Usuario)

class Atleta(models.Model):
    usuario = models.OneToOneField(Usuario,
    on_delete=models.CASCADE,
                                related_name='perfil_atleta')
    cpf = models.CharField(max_length=11, unique=True)
    categoria = models.CharField(max_length=10, #
    '5K', '10K', '21K', '42K'

    choices=[('5K', '5K'), ('10K', '10K'), ('21K', '21K'), ('42K', '42K')],
            default='10K')
    melhor_tempo = models.DurationField(null=True, blank=True)

    def __str__(self):
        return f'Atleta: {self.usuario.username} ({self.categoria})'

class Treinador(models.Model):
    usuario = models.OneToOneField(Usuario,
    on_delete=models.CASCADE,
```

```

related_name='perfil_treinador')
    cref = models.CharField(max_length=20, unique=True)
    especialidade = models.CharField(max_length=60, # ex.: 'Corrida
de Rua'
                                   blank=True)
    qtd_atletas_max = models.PositiveIntegerField(default=10)

    def __str__(self):
        return f'Treinador: {self.usuario.username} (CREf {self.cref})'

class Organizador(models.Model):
    usuario = models.OneToOneField(Usuario,
on_delete=models.CASCADE,

related_name='perfil_organizador')
    cnpj = models.CharField(max_length=14, unique=True)
    nome_organizacao = models.CharField(max_length=120)
    uf = models.CharField(max_length=2)

    def __str__(self):
        return f'Org: {self.nome_organizacao} ({self.uf})'

```

3.2 Serializers aninhados

backend/serializers.py — Cenário D

```

from django.db import transaction
from .models import Usuario, Atleta, Treinador, Organizador
from rest_framework_simplejwt.serializers import
TokenObtainPairSerializer

class AtletaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Atleta
        fields = ['cpf', 'categoria', 'melhor_tempo']

class TreinadorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Treinador
        fields = ['cref', 'especialidade', 'qtd_atletas_max']

class OrganizadorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Organizador
        fields = ['cnpj', 'nome_organizacao', 'uf']

class RegistroSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)

```

```

perfil_atleta      = AtletaSerializer(required=False)
perfil_treinador   = TreinadorSerializer(required=False)
perfil_organizador= OrganizadorSerializer(required=False)

class Meta:
    model = Usuario
    fields = ['id','username','email','password','tipo','cidade',
'perfil_atleta','perfil_treinador','perfil_organizador']

    def validate(self, data):
        tipo = data.get('tipo')
        if tipo == Usuario.Tipo.ATLETA and 'perfil_atleta' not in data:
            raise serializers.ValidationError('Atleta precisa enviar
perfil_atleta.')
        if tipo == Usuario.Tipo.TREINADOR and 'perfil_treinador' not in
data:
            raise serializers.ValidationError('Treinador precisa enviar
perfil_treinador.')
        if tipo == Usuario.Tipo.ORGANIZADOR and 'perfil_organizador' not
in data:
            raise serializers.ValidationError('Organizador precisa enviar
perfil_organizador.')
        return data

    @transaction.atomic
    def create(self, validated_data):
        d_atleta = validated_data.pop('perfil_atleta', None)
        d_treino = validated_data.pop('perfil_treinador', None)
        d_org = validated_data.pop('perfil_organizador', None)
        senha = validated_data.pop('password')
        usuario = Usuario(**validated_data)
        usuario.set_password(senha)
        usuario.save()
        if d_atleta: Atleta.objects.create(usuario=usuario, **d_atleta)
        if d_treino: Treinador.objects.create(usuario=usuario,
**d_treino)
        if d_org: Organizador.objects.create(usuario=usuario,
**d_org)
        return usuario

class LoginSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['tipo'] = user.tipo
        return token

    def validate(self, attrs):
        data = super().validate(attrs)
        user = self.user
        data['tipo'] = user.tipo
        # Dado específico do perfil na resposta do login

```

```
        if user.is_atleta() and hasattr(user, 'perfil_atleta'):
            data['categoria'] = user.perfil_atleta.categoria
        elif user.is_treinador() and hasattr(user, 'perfil_treinador'):
            data['cref'] = user.perfil_treinador.cref
        elif user.is_organizador() and hasattr(user,
            'perfil_organizador'):
            data['organizacao'] =
            user.perfil_organizador.nome_organizacao
        return data
```

2.5 Views

Atualize as Views para esse cenário.

3.4 Roteiro de testes — arquivo api.http



api.http — Cenário D (RunTrack com perfis separados)

1) Cadastrar atleta COM perfil

POST http://localhost:8000/api/signup/

Content-Type: application/json

```
{
  "username": "lucas_trail",
  "email": "lucas@runtrack.com",
  "password": "Trail@999",
  "tipo": "ATLETA",
  "cidade": "Parnamirim",
  "perfil_atleta": {
    "cpf": "12345678901",
    "categoria": "21K"
  }
}
```

2) Cadastrar treinador COM perfil

POST http://localhost:8000/api/signup/

Content-Type: application/json

```
{
  "username": "coach_roberto",
  "email": "roberto@runtrack.com",
  "password": "Coach@321",
  "tipo": "TREINADOR",
  "perfil_treinador": {
    "cref": "012345-G/RN",
    "especialidade": "Corrida de Rua e Trilha",
    "qtd_atletas_max": 15
  }
}
```

3) Cadastrar organizador COM perfil

POST http://localhost:8000/api/signup/

Content-Type: application/json

```
{
  "username": "circuito_rn",
  "email": "rn@circuito.com",
  "password": "Circuito@007",
  "tipo": "ORGANIZADOR",
  "perfil_organizador": {
    "cnpj": "99887766000100",
    "nome_organizacao": "Circuito RN de Corridas",
    "uf": "RN"
  }
}

### 4) Tentar cadastrar atleta SEM perfil_atleta (deve retornar 400)
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "sem_perfil",
  "password": "Teste@000",
  "tipo": "ATLETA"
}

### 5) Login do atleta – observe 'categoria' na resposta
POST http://localhost:8000/api/login/
Content-Type: application/json

{ "username": "lucas_trail", "password": "Trail@999" }

### 6) Login do treinador – observe 'cref' na resposta
POST http://localhost:8000/api/login/
Content-Type: application/json

{ "username": "coach_roberto", "password": "Coach@321" }

### 7) Perfil completo do atleta autenticado
GET http://localhost:8000/api/perfil/
Authorization: Bearer <access_token_do_atleta>

### 8) Treinador tenta registrar treino (403: não é atleta)
POST http://localhost:8000/api/treino/
Authorization: Bearer <access_token_do_treinador>
Content-Type: application/json

{ "distancia_km": 5, "tempo_min": 28 }
```

4. Desafio Extra — Inscrição em Evento

Implemente o endpoint POST `/api/evento/<id>/inscrever/` com as seguintes regras:

1. Só um Atleta pode se inscrever (permissão `IsAtleta`).
2. Um Treinador pode consultar a lista de inscritos do evento (GET, permissão `IsTreinador`).
3. Um Organizador pode encerrar as inscrições (PATCH, permissão `IsOrganizador`).

 Dica: crie primeiro o model `Evento` (nome, data, vagas, encerrado) e o model `Inscricao` (atleta FK, evento FK, data_inscricao). Depois construa o `ViewSet` e aplique as permissões por action.